

Report: Memory Management and Pointer Arithmetic in C

Course: Introduction to Systems Programming
Student ID: s2261xxxx Submitted: 2026-XX-XX

1. Introduction

This report discusses memory management in the C programming language, with particular focus on pointer arithmetic and the stack/heap distinction. Understanding these concepts is essential for writing efficient and secure systems-level code.

2. Stack and Heap

The stack is a region of memory that grows downward and is managed automatically by the compiler. Function-local variables are typically allocated on the stack. The heap, in contrast, is managed manually through functions such as `malloc()` and `free()`, and is used for allocations whose lifetime exceeds the enclosing scope.

3. Pointer Arithmetic

Given a pointer `p` of type `T*`, the expression `p + n` advances the pointer by `n * sizeof(T)` bytes. This scaling makes pointer arithmetic type-aware and is the foundation of array indexing in C, where `a[i]` is defined as `*(a + i)`. Careless arithmetic, however, can easily produce out-of-bounds accesses, which constitute a major class of memory safety bugs.

4. Common Pitfalls

Typical pitfalls include use-after-free, double-free, dangling pointers, and buffer overflows. Modern compilers and sanitizers (AddressSanitizer, Valgrind) help detect these at development time, but defensive coding practices remain essential.

5. Conclusion

Mastery of memory management is a core skill for systems programmers. While higher-level languages abstract these concerns away, the underlying model remains important for understanding performance and security characteristics of any program.